

SOFTWARE DESIGN COURSE

Design Patterns

Md Saeed Siddik
Assistant Professor,
Institute of IT, University of Dhaka

22

classic patterns

3

families

1994

the GoF book



Creational



Behavioral



Structural

What is a design pattern?

A typical, reusable solution to a commonly occurring problem in software design.

Think of it as a blueprint: the layout and features are fixed, but you decide how to build it in your own code.



A pattern is...

- a named, described solution
- a language-independent template
- proven across many projects



A pattern is not...

- ready-to-paste code or a library
- an algorithm (a fixed recipe of steps)
- a rule you must always follow



Recurring problem

Seen again and again in different projects



Pattern = blueprint

A named, documented solution



Your customized code

Adapted to your language and context

Why learn design patterns?

One word replaces a paragraph

WITHOUT A SHARED VOCABULARY

“Make the class hide its constructor, keep a static field with the only instance, create it on first use, and expose a static accessor...”

WITH PATTERNS

“Just make it a Singleton.”



Reusability

Apply the same idea across projects and problems instead of re-inventing it.



Standardization

A shared language for design discussions, documentation and code reviews.



Efficiency

Skip re-solving known problems and ship faster with proven approaches.



Flexibility

Abstract templates you adapt to fit your own requirements.

How to read a pattern

Every catalog entry answers the same six questions



01

Intent

What does the pattern do, in one sentence?



02

Problem

The situation and forces that call for the pattern.



03

Solution

The roles (classes/objects) and how they collaborate.



04

Structure

A class or sequence diagram of the participants.



05

Applicability

When it fits, and when it does not.

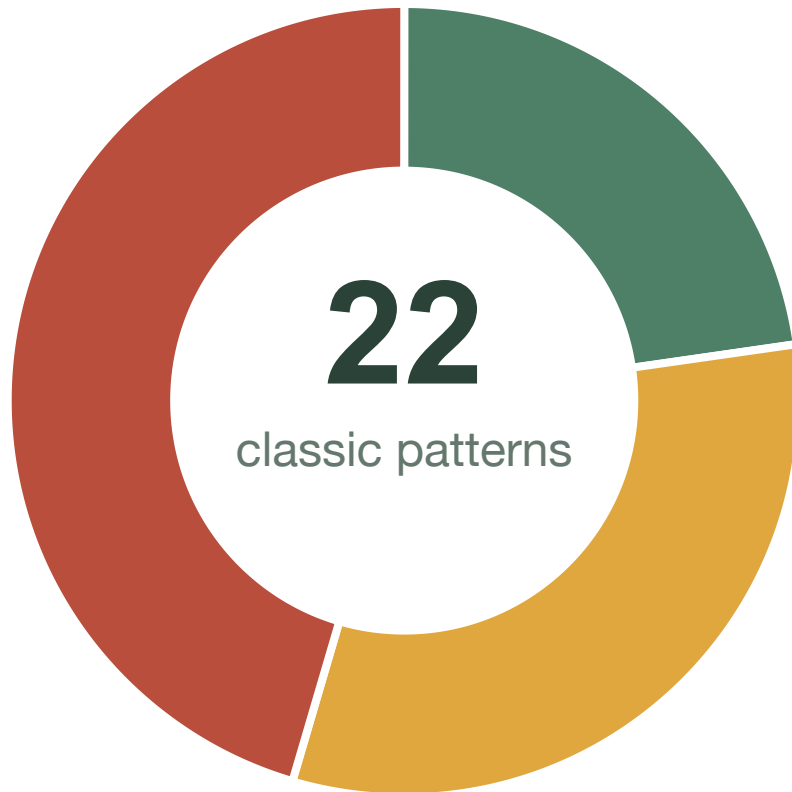


06

Consequences

Trade-offs: pros, cons, relations to other patterns.

Three families of patterns



Creational · 5

How are objects created?

Factory Method, Abstract Factory, Builder, Prototype, Singleton



Structural · 7

How are objects composed into larger structures?

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy



Behavioral · 10

How do objects communicate and share work?

Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

The catalog at a glance

Creational · 5

Factory Method , subclasses choose the class

Abstract Factory , families of related objects

Builder , step-by-step construction

Prototype , copy existing objects

Singleton , exactly one instance

Structural · 7

Adapter , make interfaces compatible

Bridge , decouple abstraction

Composite , tree treated as one object

Decorator , add behavior dynamically

Facade , one simple entry point

Flyweight , share state to save memory

Proxy , stand-in that controls access

Behavioral · 10

Chain of Responsibility , pass it along

Command , request as an object

Iterator , traverse a collection

Mediator , centralize communication

Memento , save and restore state

Observer , notify subscribers

State , behavior follows state

Strategy , swap algorithms at runtime

Template Method , skeleton + steps

Visitor , add new operations

Also documented elsewhere: Interpreter (GoF), Object Pool, Private Class Data, Null Object (SourceMaking).

Ideas behind almost every pattern



Program to an interface

Depend on abstractions, not concrete classes, so implementations can be swapped.



Favor composition over inheritance

Assemble behavior from collaborating objects instead of deep class hierarchies.



Encapsulate what varies

Isolate the parts that change so the rest of the code stays stable.

SOLID

Five design principles that make code clean, maintainable and loosely coupled, the prerequisite for everything that follows.

S

Single Responsibility

One reason to change

O

Open / Closed

Open to extend, closed to change

L

Liskov Substitution

Subtypes can replace base types

I

Interface Segregation

Small, focused interfaces

D

Dependency Inversion

Depend on abstractions

01

Creational Patterns

How objects get created, flexibly, reusably, and without tight coupling

Factory Method

Abstract Factory

Builder

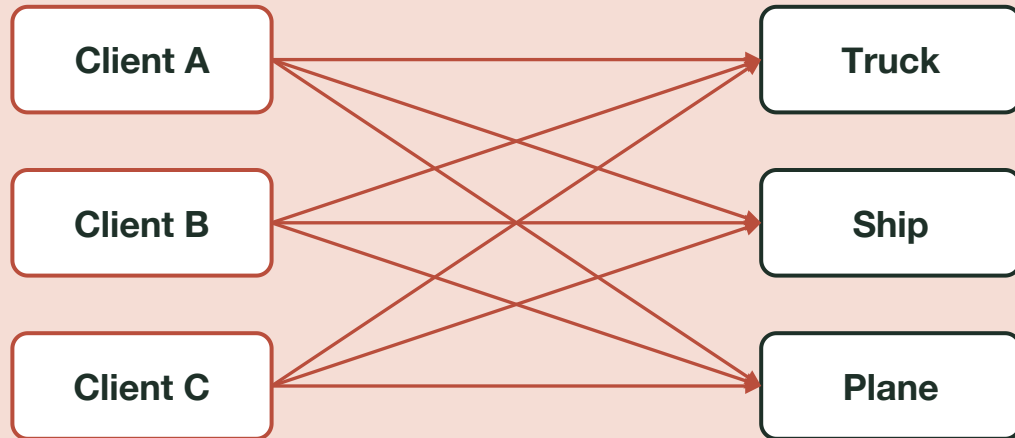
Prototype

Singleton

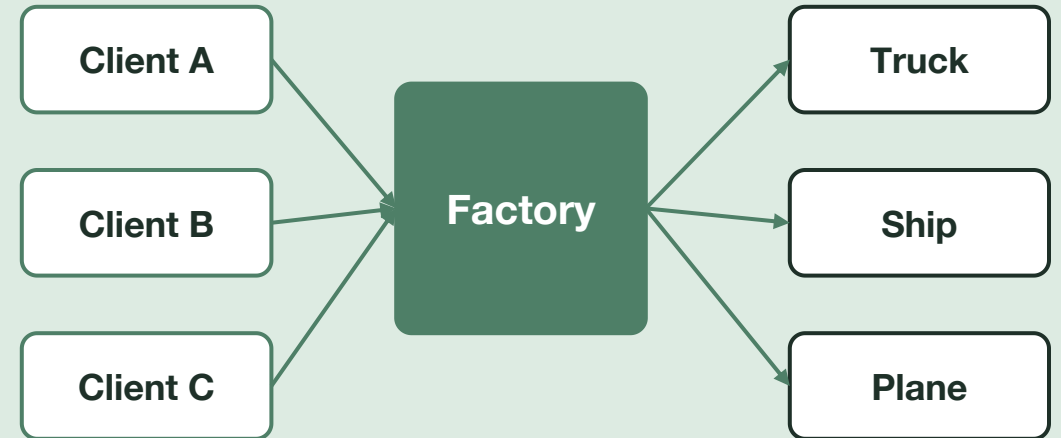


Why not just call new?

WITHOUT: every client knows every concrete class



WITH: creation lives in one place



Creational patterns decouple **what** is created from **who** uses it, so you can add new product types without rewriting clients.

Singleton

Exactly one instance, with one global access point

INTENT

Guarantee a class has only one instance and give everyone a single point of access to it.

ANALOGY

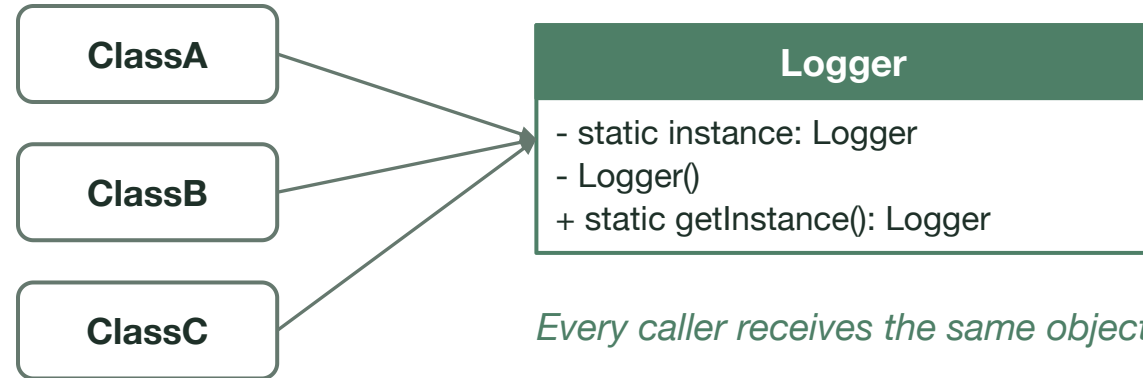
A country has one official government. Whoever is in office, “the Government” is the single point of contact.

USE WHEN

Exactly one shared resource is needed: logger, configuration, connection pool.

WATCH OUT

Global state, thread-safety and hard-to-test code.

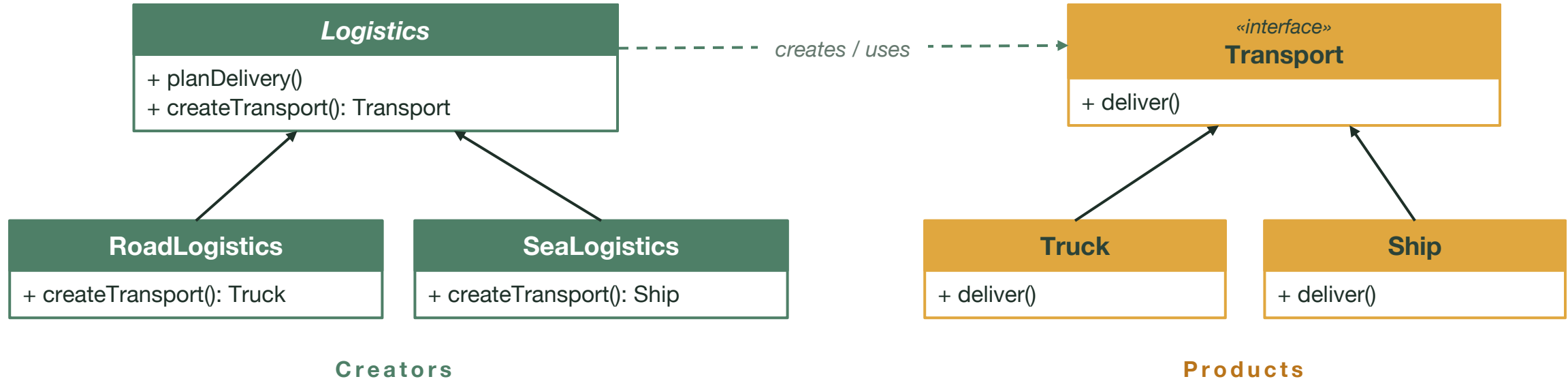


JAVA

```
public class Logger {  
    private static Logger instance;  
  
    private Logger() { } // no outside "new"  
  
    public static synchronized Logger getInstance() {  
        if (instance == null) instance = new Logger();  
        return instance;  
    }  
}
```

Factory Method

Let subclasses decide which class to instantiate



USE WHEN

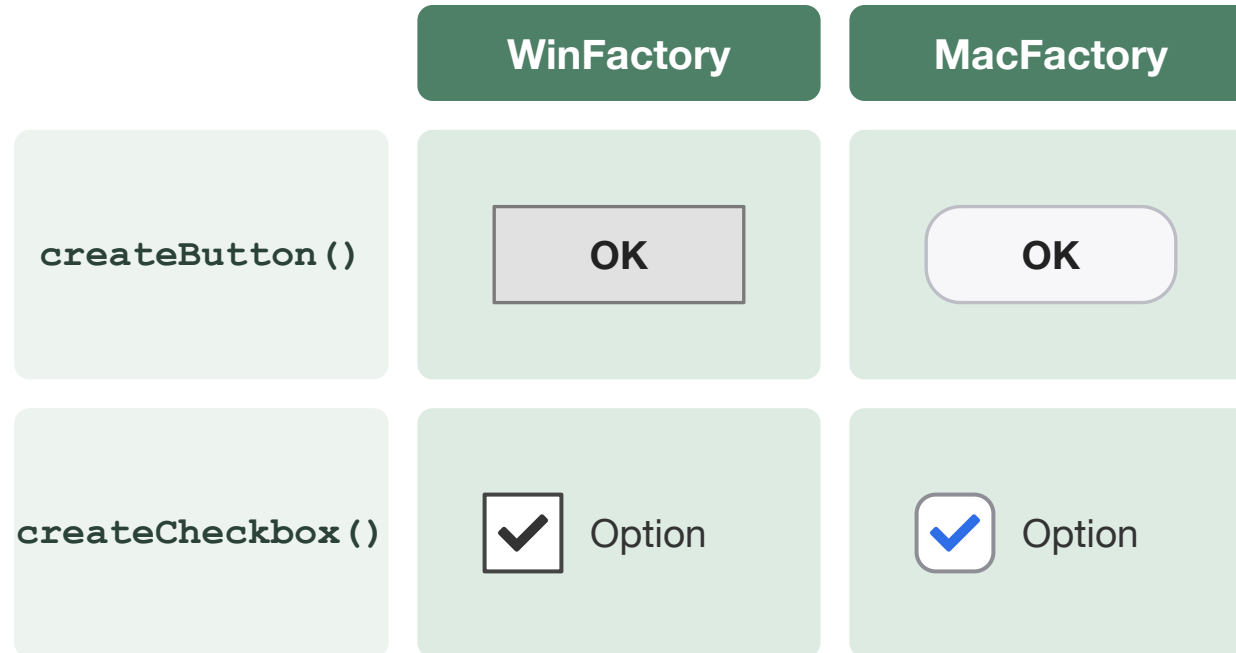
You cannot know the exact classes beforehand, or you want callers to add new product types without touching client code.

```
abstract class Logistics {  
    abstract Transport createTransport();  
    void planDelivery() {  
        createTransport().deliver();  
    }  
}
```

JAVA

Abstract Factory

Create whole families of related objects that always match



Products from one factory always belong together, mixing a Windows button with a Mac checkbox is impossible by design.

INTENT

Produce families of related objects without naming their concrete classes.

USE WHEN

Your code must work with several product families (themes, platforms, databases) that must not be mixed.

JAVA

```
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class MacFactory implements GUIFactory {...}

GUIFactory f = new MacFactory();
Button b = f.createButton();           // Mac
Checkbox c = f.createCheckbox();       // Mac
```

Builder

Construct complex objects step by step

Telescoping constructor, what does each argument mean?

BEFORE

```
Pizza p = new Pizza(  
    12,        // size?  
    true,      // cheese?  
    false,     // pepperoni?  
    true,      // mushrooms?  
    false);    // extra sauce?
```

Fluent builder, every step has a name

AFTER

```
Pizza p = new Pizza.Builder()  
    .size(12)  
    .cheese()  
    .mushrooms()  
    .build();           // validated, immutable
```



Readable calls

Named steps replace a wall of unlabeled boolean and numeric arguments.



Immutable results

The object is handed out complete via `build()`, so it can be immutable.



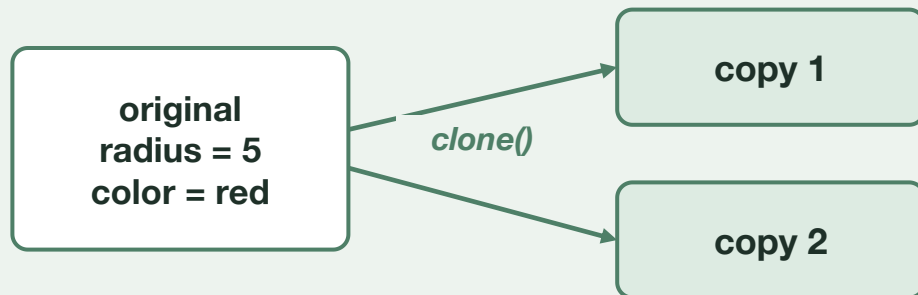
Many representations

One construction process can yield different products (e.g., via a Director).

Prototype & Object Pool

Prototype

Create new objects by copying an existing one, like a cell dividing in two.

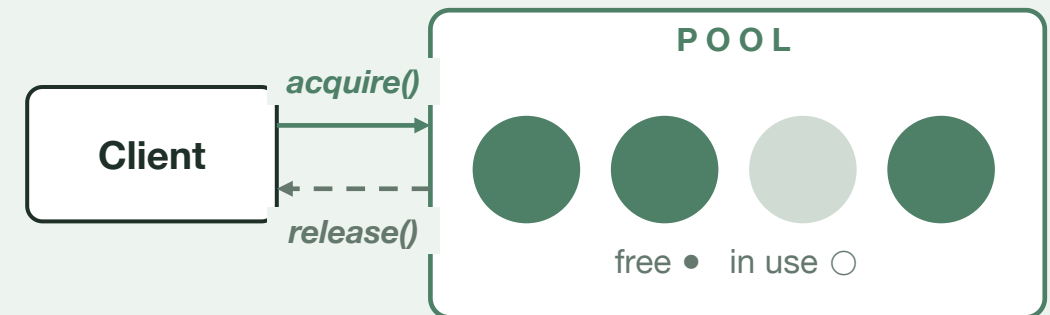


JAVA

```
Shape copy = original.clone();  
copy.setColor("blue"); // original unchanged  
  
// avoids re-running expensive setup
```

Object Pool

Recycle expensive objects (DB connections, threads) instead of creating and destroying them.



JAVA

```
Connection c = pool.acquire();  
try { c.query(sql); }  
finally { pool.release(c); }
```

02

Structural Patterns

How classes and objects are assembled into larger, flexible structures

Adapter

Bridge

Composite

Decorator

Facade

Flyweight

Proxy



Seven patterns, seven analogies

Assemble objects into bigger structures without tangling them



Adapter

A travel plug converts one socket shape into another.



Bridge

A universal remote controls any kind of device.



Composite

Folders hold files and other folders, all handled the same way.



Decorator

Coffee toppings: each layer wraps the one before it.



Facade

A hotel concierge: one request triggers many back-office services.



Flyweight

Thousands of trees share one species definition; only position differs.

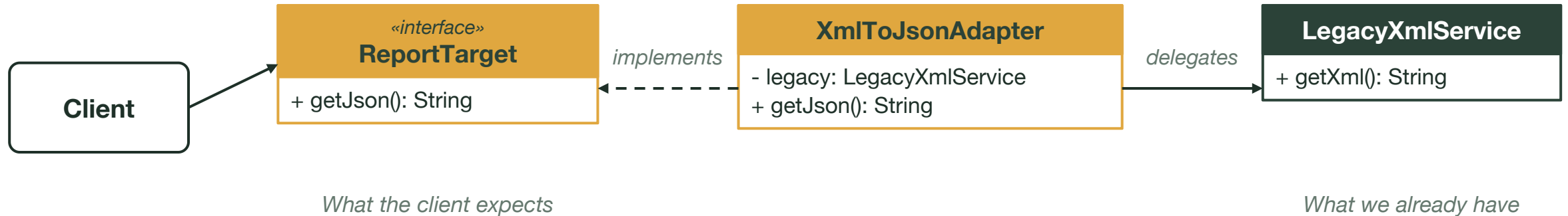


Proxy

A credit card stands in for cash and guards access to your bank.

Adapter

Make incompatible interfaces work together



INTENT

Convert one class's interface into the one clients expect, without changing either side.

SEEN IN

Arrays.asList() (array → List) and InputStreamReader (bytes → characters).

USE WHEN

You must reuse a legacy or third-party class with the wrong interface.

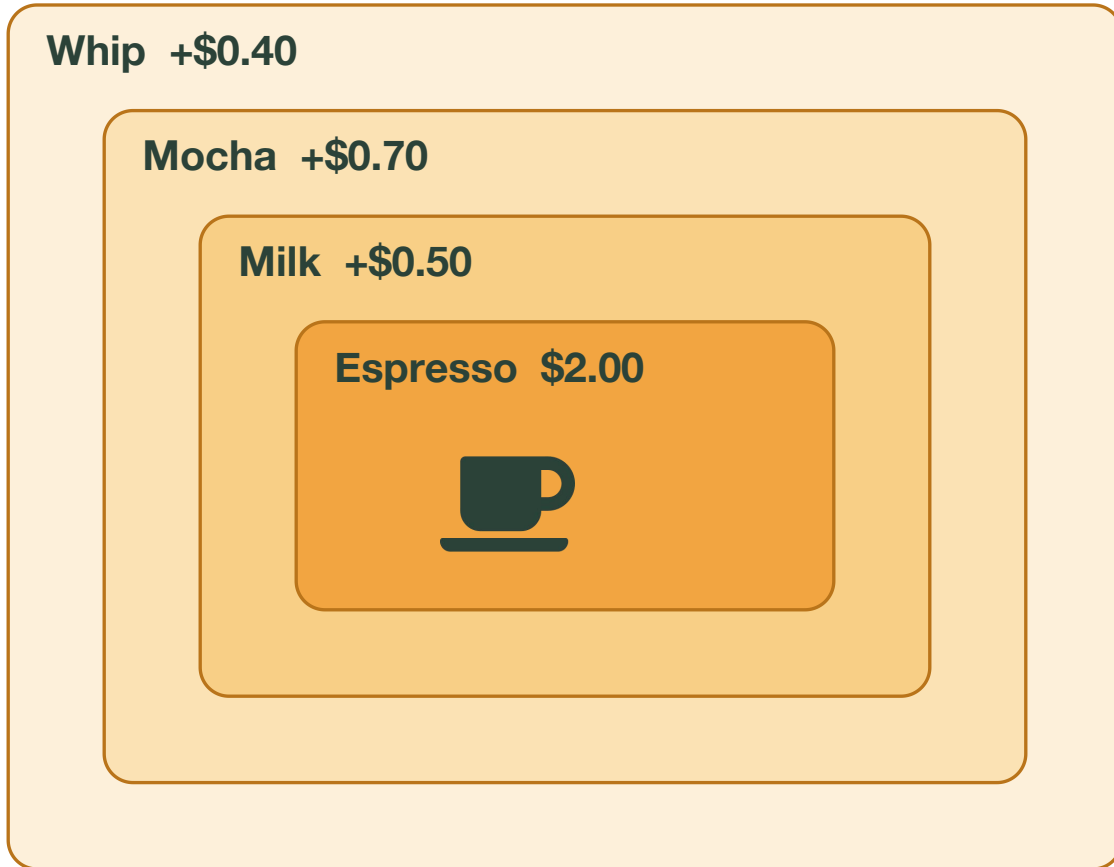
```

JAVA
class XmlToJsonAdapter implements ReportTarget {
    private LegacyXmlService legacy;

    public String getJson() {
        String xml = legacy.getXml(); // old format
        return Converter.xmlToJson(xml); // new format
    }
}
  
```

Decorator

Add responsibilities by wrapping objects



cost() = 2.00 + 0.50 + 0.70 + 0.40 = \$3.60

INTENT

Attach new behavior to an object dynamically by wrapping it in objects that share its interface.

WHY

Subclassing every combination (milk + mocha + whip...) explodes into dozens of classes. Wrapping composes behavior at runtime.

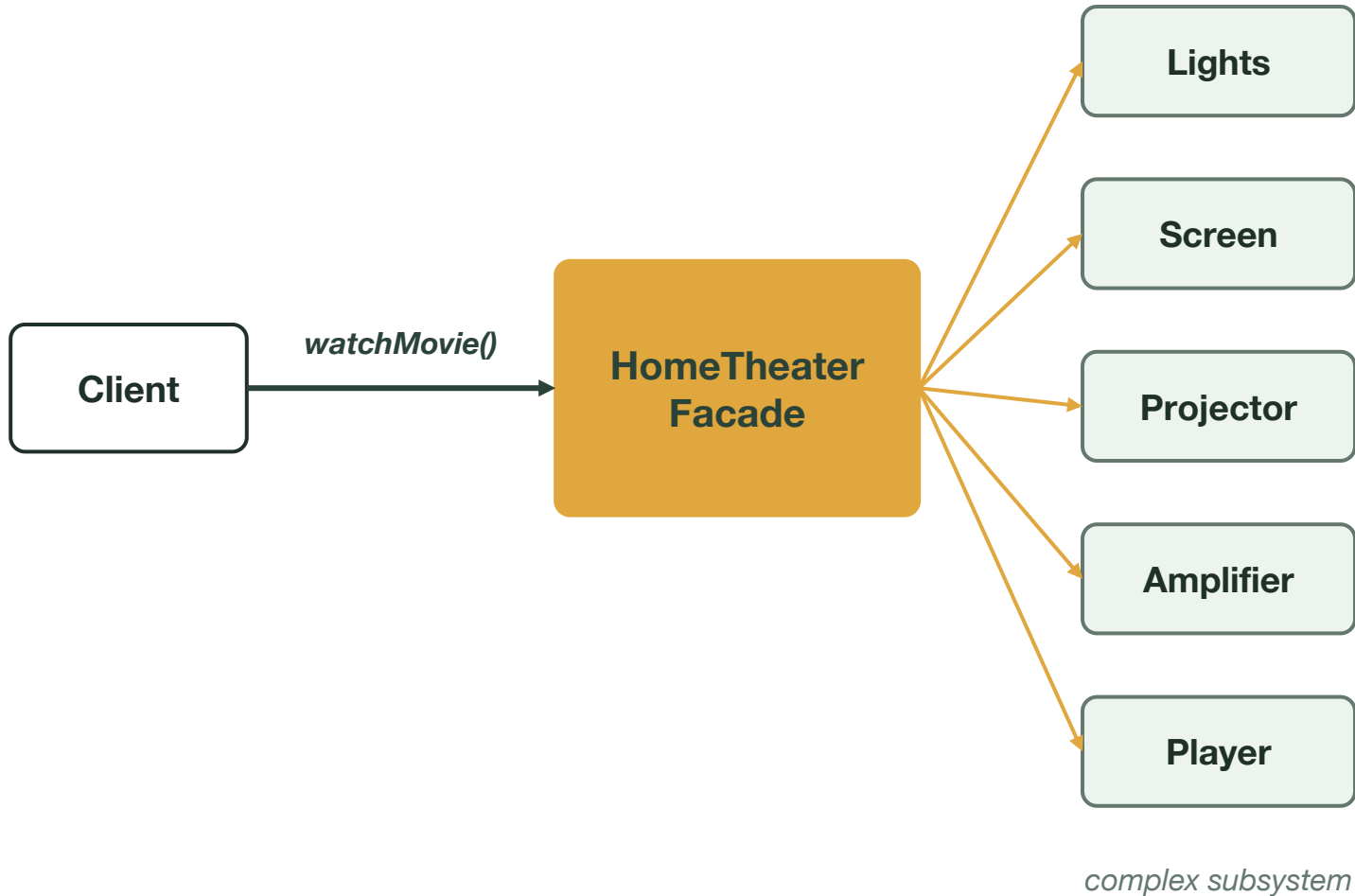
JAVA

```
Coffee c = new Espresso();           // $2.00
c = new Milk(c);                     // + $0.50
c = new Mocha(c);                    // + $0.70
c = new Whip(c);                     // + $0.40
System.out.println(c.cost());        // 3.60
```

Seen in: `new BufferedReader(new FileReader(path))`

Facade

One simple entry point to a complex subsystem



5 → 1

calls the client has to make

INTENT

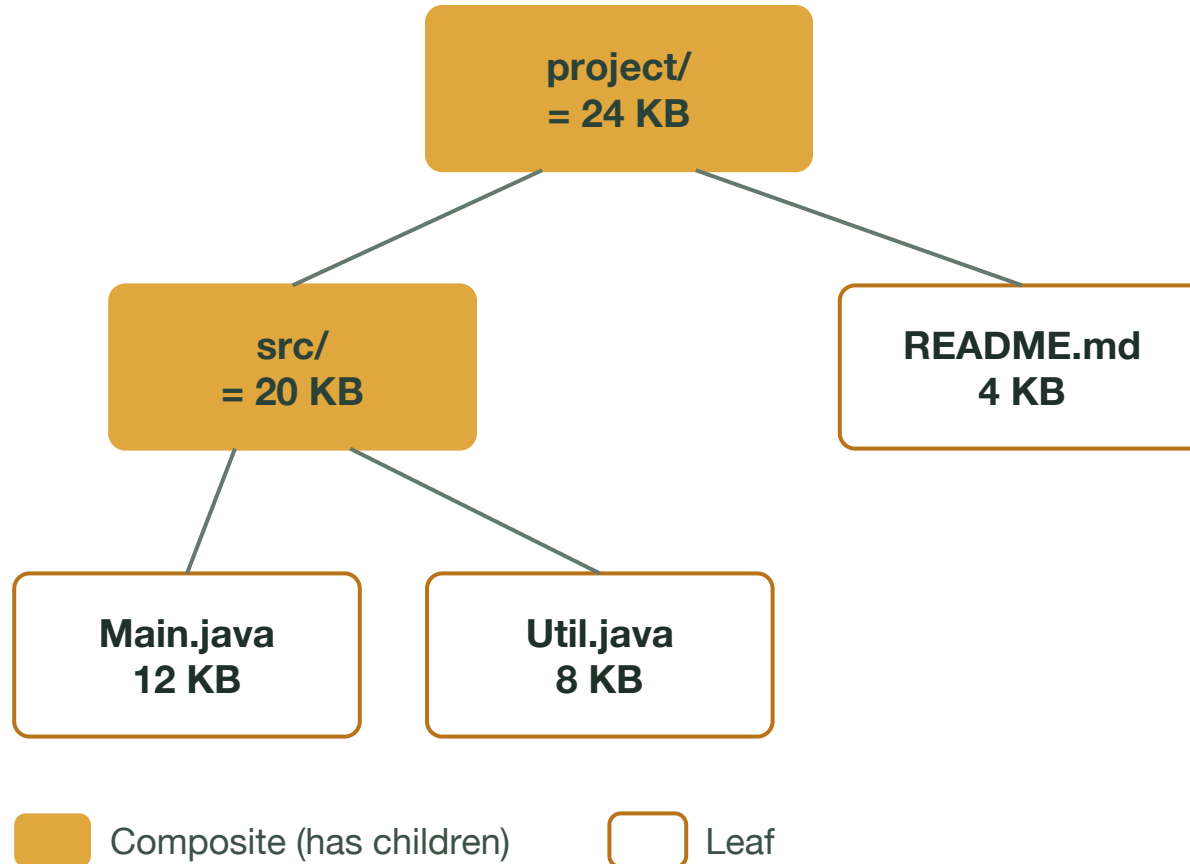
Provide a simple interface to a library, framework or set of classes.

JAVA

```
class HomeTheaterFacade {
    void watchMovie(String t) {
        lights.dim(10);
        screen.down();
        projector.on();
        amp.setVolume(5);
        player.play(t);
    }
}
```

Composite

Treat single objects and groups uniformly



INTENT

Compose objects into tree structures and let clients treat leaves and groups through one interface.

USE WHEN

Your data is naturally a tree: file systems, UI widgets, org charts, menus.

JAVA

```
interface Item { long size(); }

class File implements Item {
    public long size() { return bytes; }
}

class Folder implements Item {
    List<Item> children;
    public long size() {
        return children.stream()
            .mapToLong(Item::size).sum();
    }
}
```

Proxy, Bridge & Flyweight



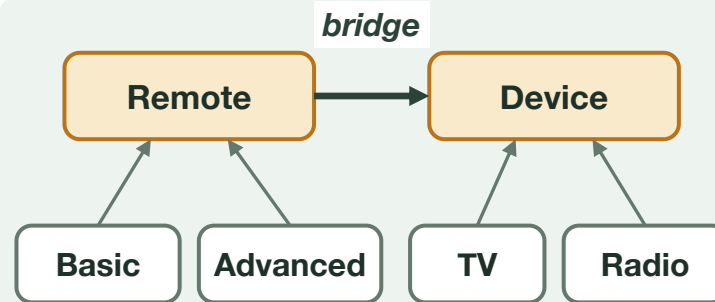
cache · lazy-load · access check

Proxy

A stand-in that controls access to the real object: lazy loading, caching, permission checks, logging.

SEEN IN

Spring AOP, Hibernate lazy-loading proxies

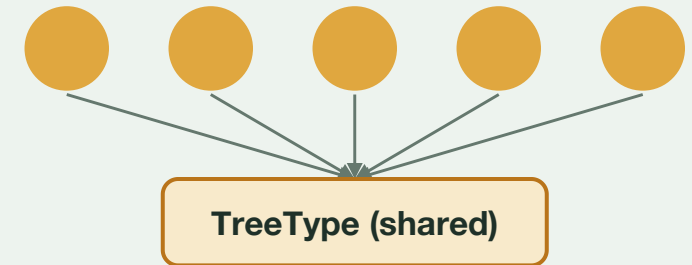


Bridge

Split a class into an abstraction and an implementation hierarchy so each can vary on its own.

SEEN IN

Shapes + drawing APIs; remotes + devices



Flyweight

Share the common (intrinsic) state between many objects to save memory; keep only the unique state per object.

SEEN IN

Java String pool, Integer.valueOf() cache

03

Behavioral Patterns

How objects communicate, cooperate and share responsibility

Strategy

Observer

Command

State

Chain of Responsibility

Template Method

Iterator

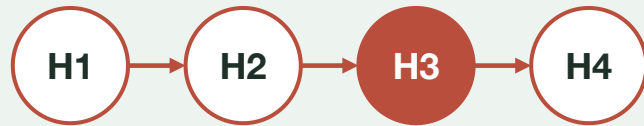
Mediator

Memento

Visitor



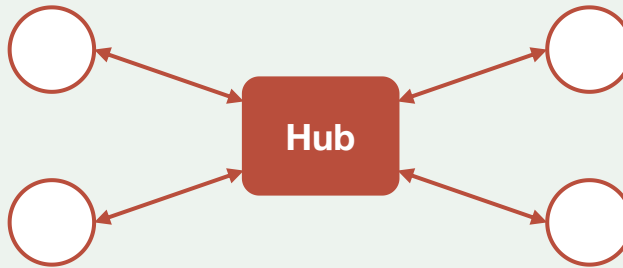
How objects talk to each other



request → handled by H3

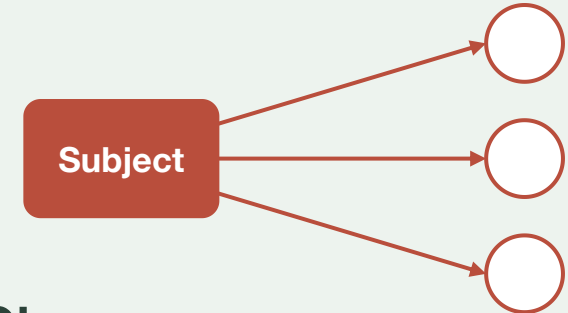
Chain of Responsibility

Pass a request along a chain of handlers until one of them deals with it.



Mediator

Objects talk through a central hub instead of directly to each other, like air-traffic control.



Observer

One change is announced to many subscribers, who react on their own.

BEHAVIOR AS AN OBJECT, pass it around, swap it, store it

Strategy

Command

State

Template Method

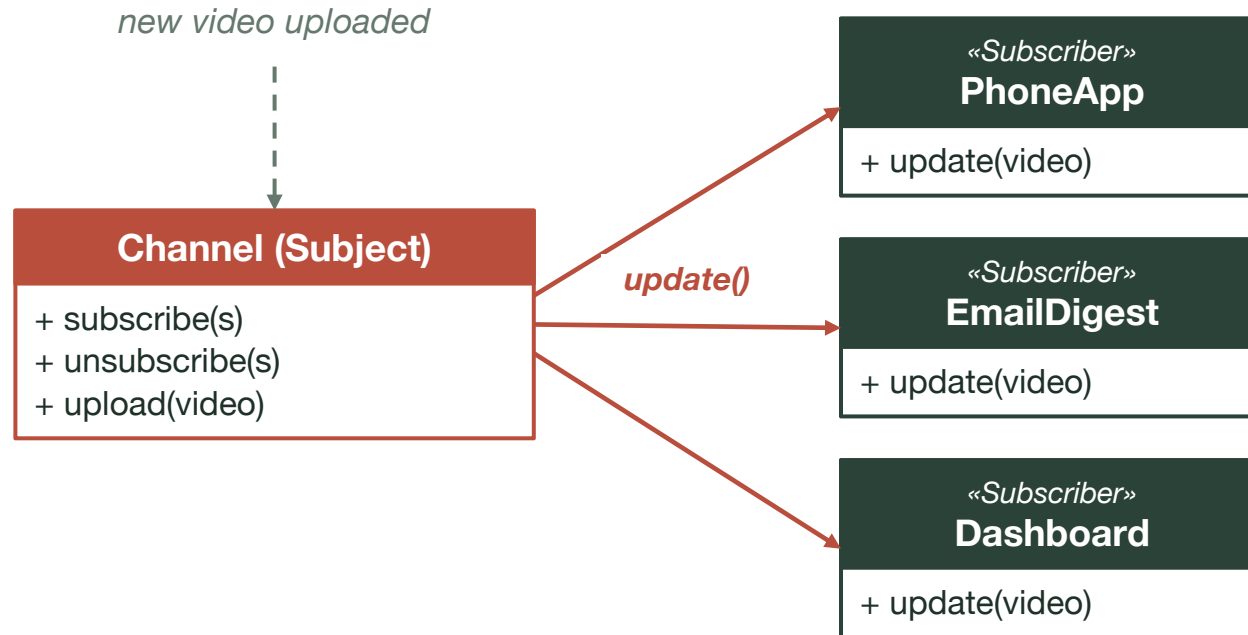
Visitor

Iterator

Memento

Observer

Notify many objects when one object changes



Analogy: a newspaper subscription. Subscribe and every issue arrives; cancel and it stops, the publisher never needs to know who is reading.

INTENT

Define a one-to-many dependency so that dependents are notified automatically when the subject changes.

```
JAVA
interface Subscriber { void update(String v); }

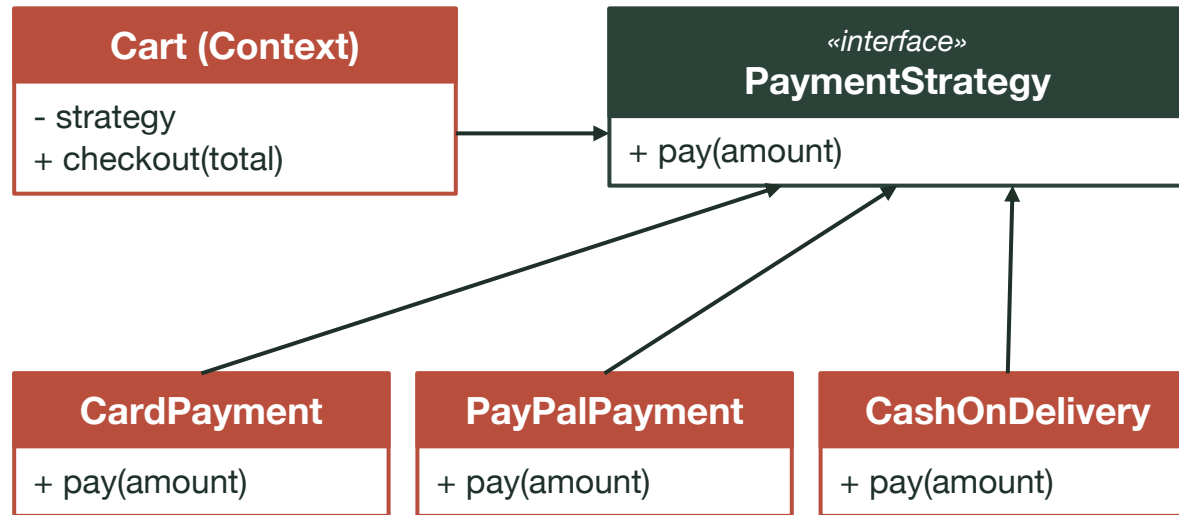
class Channel {
    List<Subscriber> subs = new ArrayList<>();

    void subscribe(Subscriber s) { subs.add(s); }
    void upload(String v) {
        subs.forEach(s -> s.update(v));
    }
}
```

Seen in: DOM `addEventListener`, Redux/React subscriptions, pub/sub messaging

Strategy

Swap algorithms at runtime through a common interface



BEFORE

```

if (type == "card") ...
else if (type == "paypal") ...

```

Every new payment method means editing Cart.

INTENT

Define a family of algorithms, put each in its own class, and make them interchangeable.

USE WHEN

You want to switch behavior at runtime or replace big conditionals with polymorphism.

```

JAVA
interface PaymentStrategy { void pay(double a); }

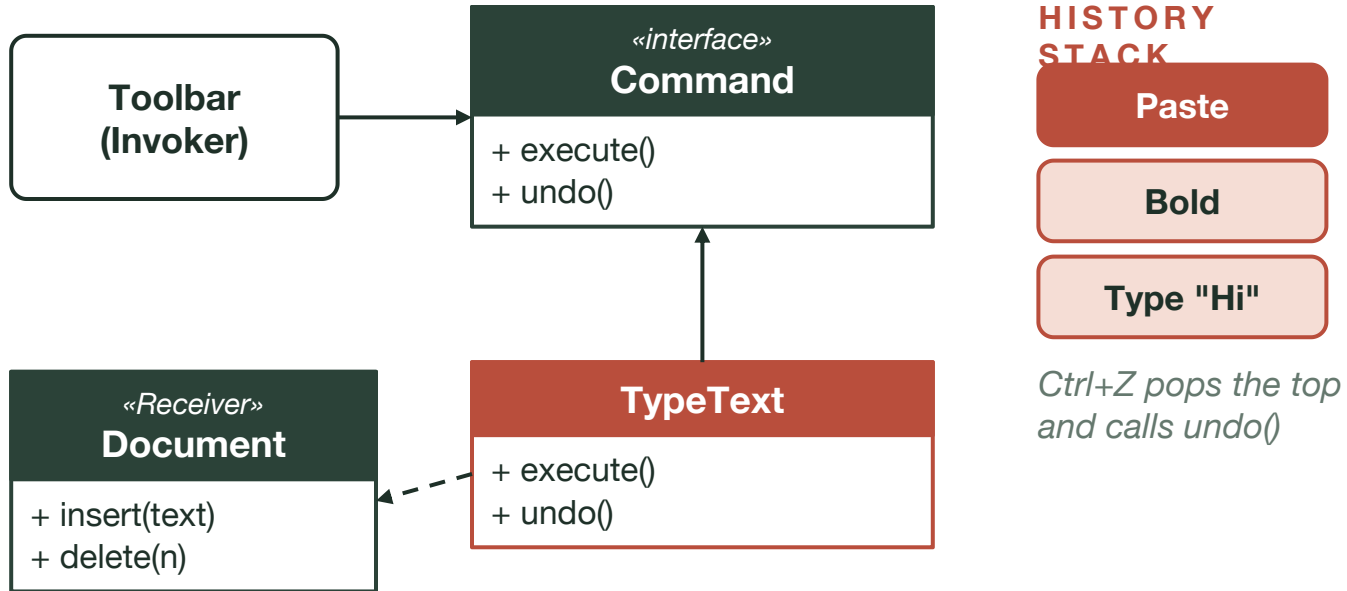
class Cart {
    PaymentStrategy strategy;
    void setStrategy(PaymentStrategy s) {
        strategy = s;
    }
    void checkout(double t) { strategy.pay(t); }
}

cart.setStrategy(new PayPalPayment());

```

Command

Turn a request into an object, enabling undo, queues and logs



INTENT

Encapsulate a request as an object so it can be stored, queued, logged and undone.

JAVA

```

class TypeText implements Command {
    Document doc; String text;

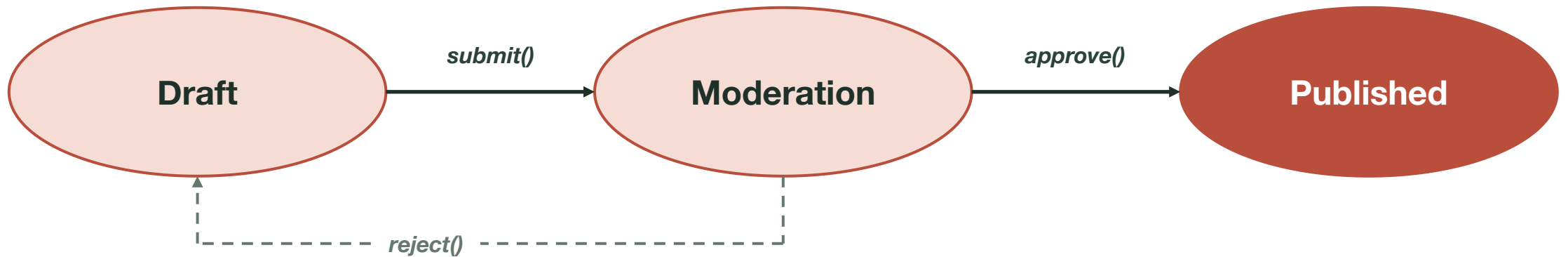
    public void execute() {
        doc.insert(text);
    }
    public void undo() {
        doc.delete(text.length());
    }
}

history.push(cmd); cmd.execute();
history.pop().undo(); // Ctrl+Z
  
```

Use when: undo/redo, queuing or scheduling operations, macro recording, transactional operations, remote calls.

State

An object changes its behavior when its internal state changes



INTENT

Let an object alter its behavior when its state changes.

ANALOGY

A vending machine acts differently when idle, holding coins or dispensing.

State vs Strategy: states switch themselves; a client picks a strategy.

JAVA

```
interface State { void next(Doc d); }

class Draft implements State {
    public void next(Doc d) { d.setState(new Moderation()); }
}

class Moderation implements State {
    public void next(Doc d) { d.setState(new Published()); }
}

// Doc delegates: state.next(this);
```

More behavioral patterns



Chain of Responsibility

Support tickets escalate from level 1 to level 2 to a manager until someone handles them.

Seen in: Servlet filters, middleware



Template Method

A recipe skeleton: subclasses fill in the steps that differ.

Seen in: `setUp()` / `tearDown()` hooks



Iterator

Walk through a collection without knowing how it is stored.

Seen in: The for-each loop



Mediator

Air-traffic control: planes talk to the tower, not to each other.

Seen in: Chat rooms, UI dialogs



Memento

A save-game snapshot lets you restore an earlier state.

Seen in: Undo history, snapshots



Visitor

Apply a new operation across different object types without editing their classes.

Seen in: Compilers, AST walkers

You already use patterns

Pattern	Where you have already met it
Singleton	<code>Runtime.getRuntime()</code> ; application-wide loggers and configuration objects
Factory Method	<code>Calendar.getInstance()</code> , <code>List.of(...)</code> , framework “create...” hooks
Builder	<code>HttpRequest.newBuilder()</code> , Lombok <code>@Builder</code>
Adapter	<code>Arrays.asList()</code> (array → List), <code>InputStreamReader</code> (bytes → characters)
Decorator	<code>new BufferedReader(new FileReader(path))</code> and other java.io wrappers
Proxy	Spring AOP and Hibernate lazy-loading proxies
Observer	Browser <code>addEventListener</code> ; React / Redux state subscriptions
Strategy	A Comparator passed to <code>Collections.sort()</code>
Iterator	The for-each loop over any Iterable

Choosing the right pattern

Start from the pain you feel, not from the pattern name

Creating objects is painful?

Need exactly one shared instance

→ **Singleton**

Constructor with 10 optional parameters

→ **Builder**

Class to create is unknown until runtime

→ **Factory Method**

Wiring objects is painful?

An old API does not match the new interface

→ **Adapter**

Adding features causes subclass explosion

→ **Decorator**

Client drowns in subsystem calls

→ **Facade**

Object interaction is painful?

Swap algorithms at runtime

→ **Strategy**

Many objects must react to one change

→ **Observer**

Need undo/redo or queued requests

→ **Command**

Use patterns wisely

16 / 23

GoF patterns that Peter Norvig showed are simplified or eliminated by language features in Lisp or Dylan.

Rule of thumb

Start simple (KISS, YAGNI). Refactor toward a pattern when real pain appears: duplicated code, ripple-effect changes.



Over-engineering

When you own a hammer, everything looks like a nail. Do not add patterns “just in case”.



Language workarounds

Some patterns compensate for missing language features, such as first-class functions.



Copy-paste adoption

Applying a pattern unchanged, without adapting it, can add duplication and inefficiency.



Fuzzy foundations

Critics call patterns ad hoc and not always distinct from ordinary abstraction, MVC predates them.

Class activity: refactor the notifier

Work in pairs · 15 minutes · sketch the classes before coding

BEFORE

```
class Notifier {  
    void send(String type, String msg, User u) {  
        if (type.equals("email")) {  
            // connect SMTP, format HTML ...  
        } else if (type.equals("sms")) {  
            // call SMS gateway ...  
        } else if (type.equals("push")) {  
            // call push service ...  
        }  
        log(msg);           // copy-pasted everywhere  
    }  
}
```

HINTS

Strategy

Observer

Decorator

Factory Method

1

Spot the smells

Which code must change every time we add a new channel?

2

Extract the channels

Put each channel behind an interface, which pattern?

3

React to events

Let users subscribe to the events they care about.

4

Add cross-cutting behavior

Add logging and retry without touching any channel class.

Key takeaways



- 1 Patterns are blueprints, named solutions you adapt, not code you paste.
- 2 Three families: Creational (create), Structural (compose), Behavioral (communicate).
- 3 Learn the intent and the problem first; the UML comes second.
- 4 Program to interfaces, favor composition, encapsulate what varies.
- 5 Reach for a pattern to relieve real pain, not to decorate code.

Resources & next steps



Refactoring.Guru

Illustrated catalog with analogies, structure diagrams and code in many languages.

refactoring.guru/design-patterns



SourceMaking

Concise pattern descriptions, plus anti-patterns and refactoring guides.

sourcemaking.com/design_patterns



THE ORIGINAL Design Patterns: Elements of Reusable Object-Oriented Software

Gamma, Helm, Johnson, Vlissides (1994)

Content summarised in original wording from the three sources above; all diagrams were drawn for this deck.